# SyntaxLens

## Multi-Input Syntactical Analyzer: User Manual and System Documentation

**COM 241 Final Project** · October 2026 · Group members: Marco Andrei R. Belen · Gabriel M. Caporado · Ariz Luiz L. Guillarte · Brix Martin M. Paña

Live app: https://multi-input-syntactical-analyzer-ap.vercel.app

Source code: https://github.com/codrei/Multi-Input-Syntactical-Analyzer-Application-Project-in-Programming-Language

## 1. Overview

SyntaxLens checks source code for syntax errors **without running it**. It splits the code into tokens, checks them against syntax rules, and prints a report that either confirms the code is valid or lists each error with its line, column, category, explanation and a suggested fix.

- **Three input modes:** a single line, a block of code, or an uploaded file.
- **Five languages:** Python, Java, C, C++ and C#, detected automatically (or chosen by the user).
- **Eight syntax checks:** the six in the specification (brackets, string literals, statement terminators, operators, control structure headers, identifier names) plus indentation & block structure, and illegal characters & malformed literals.
- **Two interfaces, one engine:** a web app (online, or offline on your own computer) and a command line. Both call the same analysis function, so they always agree.
- **Reports:** the specification's exact format (classic), or a detailed report with error codes, a pointer to the exact column, fix suggestions and totals for every category; also as JSON data.

## 2. Getting Started

| Way to run | How | What you need |
|---|---|---|
| Online | Open multi-input-syntactical-analyzer-ap.vercel.app | A browser and internet |
| Windows, offline | Double-click `run.bat` in the project folder; the app opens in the browser | Python 3.10 or newer |
| Command line | `py -m syntaxlens` (menu) or `py -m syntaxlens file code.py` | Python 3.10 or newer |

Python is free from python.org (tick "Add python.exe to PATH" while installing). SyntaxLens uses only Python's standard library, so nothing else has to be installed. On macOS or Linux, write `python3` instead of `py`.

## 3. Using the Three Input Modes

### 3.1 In the web app

1. **Language:** leave it on *Auto-detect*, or choose the language in the menu at the top.
2. **Mode 1, Single Line:** type one statement, e.g. `x = 10` or `System.out.println("Hello");`, and press Enter. The example buttons fill in ready-made lines.
3. **Mode 2, Code Block:** type or paste several lines into the editor and press *Analyze* (or Ctrl+Enter). The *Load a sample* menu offers the specification's examples and one demo file per language. *Analyze as I type* re-checks the code while you edit.
4. **Mode 3, Upload File:** drag a file onto the box, or click it to choose one (.py, .java, .c, .cpp, .h, .cs, .txt; up to 2 MB). Empty, binary (e.g. images) and oversized files are refused with a message.

The results show a PASSED / FAILED banner, counts, and bar charts of errors per check and tokens per category. Errors are marked in the code with a red line, a dot in the margin and a wavy underline. The tabs give the **Errors** (click one to jump to its line), the **Line Breakdown**, all **Tokens** (filter by category) and the text **Report** (detailed or classic) with Copy and Download (.txt, .json).

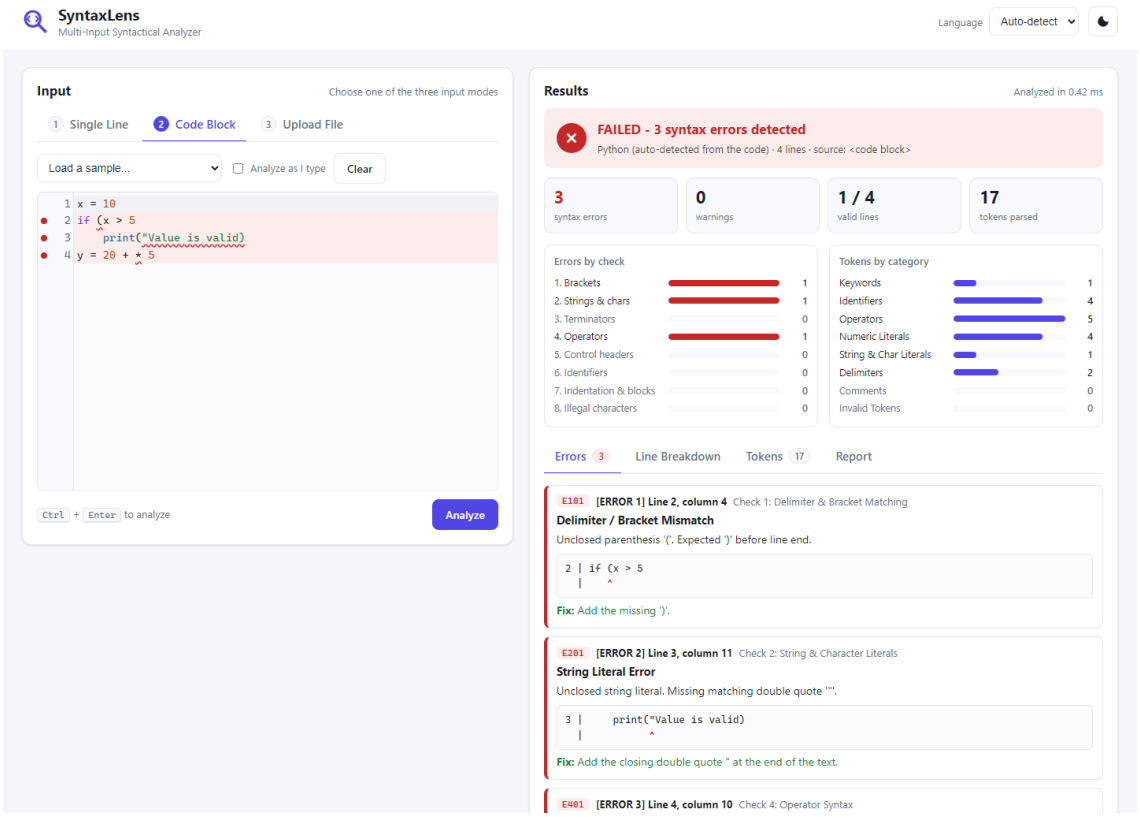



Figure 1. The specification's Example 2 in the web app: three errors, each marked in the code and explained on the right.

## 3.2 On the command line

| Input mode | Command |
|---|---|
| Menu (all modes) | `py -m syntaxlens`, then choose 1, 2 or 3 |
| 1. Single line | `py -m syntaxlens line "x = 10"` |
| 2. Code block | `py -m syntaxlens block`, paste the code, end with a line holding only `.` |
| 3. File | `py -m syntaxlens file samples\spec_example_2.py` (several files are allowed) |

Options: `--lang python|java|c|cpp|csharp`, `--style classic` (the specification's exact layout), `--format json`, `--output report.txt`, `--no-color`. The exit code is 0 for valid code, 1 when syntax errors are found and 2 when the input cannot be read.

## 3.3 Reading an error

Each error names its line, category and cause. The detailed style adds the error code, the check that found it, a caret under the column and a fix:

```
[ERROR 3] Line 4: y = 20 + * 5
  - Category: Invalid Operator Sequence
  - Details: Consecutive binary operators '+ *' without an operand in between.
  - Location: line 4, column 10 (check 4: Operator Syntax, code E401)
      4 | y = 20 + * 5
        |          ^
  - Fix: Remove one of the operators, or put a value between them.
```

## 4. System Architecture

SyntaxLens is a pipeline of seven small stages, each in its own Python module. The output of one stage is the input of the next:

| Stage | Module | What it does |
|---|---|---|
| 1. Source | `source.py` | Decodes the input (UTF-8, Windows-1252, byte-order marks), turns Windows line endings into one form, refuses blank, binary and oversized input. |
| 2. Detect | `detect.py` | Chooses the language: the user's choice, else the file extension, else weighted clues in the code (e.g. `System.out.println` for Java). |
| 3. Lexer | `lexer.py` | Splits the text into tokens with one regular expression and sorts them into 8 categories; marks lexical problems on the tokens. |
| 4. Statements | `structure.py` | Groups tokens into statements (Python: logical lines and indentation; C family: `;` `{` `}`) and labels each one. |
| 5. Checks | `checks/` | Eight independent modules. Each error type belongs to exactly one check, so nothing is reported twice. |
| 6. Filter | `analyzer.py` | Keeps the root cause of each mistake and drops the follow-on errors it causes in the same statement. |
| 7. Report | `report/`, `cli.py`, `web/` | Text report (classic or detailed), JSON, the command line and the web app. |

**Languages are data, not code.** Each language is a *profile* (`profiles/`) with its spelling (keywords, operators, comments, number and string formats) and its `LanguageRules`: switches for what it allows, such as pointers in C or properties in C#. The shared code never asks which language it is analyzing, and a test enforces this. The web app is a small WSGI application from Python's standard library; it calls the same analysis function as the command line and runs both locally and on Vercel.

## 5. Parsing Logic

### 5.1 Tokenizing

All token patterns of a language are joined into **one master regular expression**, one named group per token type. Starting at the first character, the lexer asks which group matches, records that token and continues after it: a single left-to-right pass. Three rules make it reliable:

- **Order matters:** comments and strings are tried before operators (`//` is a comment, not two slashes) and operators longest first (`**=` before `**` before `*`).
- **Strings and comments are single tokens,** so code-like text inside them, such as `"if (x > 0)"`, is never checked as code.
- **It never stops:** a character the language does not allow becomes an *invalid* token, and an unclosed string ends at the end of its line.

Every token falls into one of 8 categories, which the report lists with totals: keywords, identifiers, operators, numbers, strings and characters, delimiters, comments, and invalid tokens. The specification's Example 1 gives exactly 14 tokens.

### 5.2 The eight checks

| # | Check | How it works | Example error |
|---|---|---|---|
| 1 | Delimiters & brackets | A stack: each opening bracket is pushed; a closing bracket must match the one on top. Leftovers were never closed. | `if (x > 5` |
| 2 | String & char literals | The lexer marks literals with no closing quote; a char literal must hold one character. | `print("hi)` |

| | | | |
|---|---|---|---|
| 3 | Statement terminators | Missing `;` in Java, C, C++ and C#; missing `:` after a Python header. | `int x = 10` |
| 4 | Operator syntax | A two-state machine that expects a value or an operator; `+5` is a sign, `+ *` is an error. | `y = 20 + * 5` |
| 5 | Control headers | A small grammar per header: if, for, while, switch, def, class. | `for i range(10):` |
| 6 | Identifier naming | Names start with a letter or _ and are not reserved words. | `int 2nd = 3;` |
| 7 | Indentation & blocks | A stack of indentation levels (as in CPython); `else`, `case`, `break` must be inside the right block. | unexpected indent |
| 8 | Illegal characters & literals | Illegal characters, malformed numbers, unclosed comments, curly quotes. | `3.14.15` |

### 5.3 Error recovery

The analyzer does not stop at the first error. (1) The lexer never stops. (2) At each line break inside an unfinished statement it asks whether the line ends with a complete value while the next line starts something new; if so, the statement ends there and the missing `)` or `;` is reported on this line. (3) A filter keeps only the root cause: an unclosed string hides the bracket and operator errors it caused in the same statement. This is why the specification's Example 2 gives exactly its three errors. Python's own parser reports only the first one, and a naive checker reports about six.

## 6. Regular Expressions and Libraries

| Token | Pattern (simplified) |
|---|---|
| Identifier (Python, C, C++) | `[^\W\d]\w*` (a letter or _, then letters, digits or _) |
| Identifier (Java) | `(?:[^\W\d]\|\$)(?:\w\|\$)*` (also allows $) |
| Number (Python) | `0[xX](?:_?[0-9a-fA-F])+ \| [1-9](?:_?[0-9])* \|` floats, from CPython's tokenizer |
| String | `"(?:[^"\\\n]\|\\.)*"` unclosed: the same without the closing quote |
| Comments | `#[^\n]* //[^\n]* /\*[\s\S]*?\*/` |
| Operators | one alternation, longest first: `\*\*=\|//=\|==\|\+\|-\|...` |
| Anything else | `.` (one illegal character becomes an invalid token) |

SyntaxLens uses only Python's standard library (`re`, `dataclasses`, `enum`, `wsgiref`). No parser generator or AST library is used: they stop at the first error, so they cannot produce the specification's Example 2. Python's `tokenize` module is used only in the tests, to confirm that our Python lexer produces the same tokens. The web editor is CodeMirror 5 (MIT license), stored in the repository so the app works offline.

## 7. Testing

- **606 automated tests** run on Windows and Linux with Python 3.10 to 3.14 on every change.
- Both example reports of the specification are reproduced exactly, word for word.
- **No false alarms:** 19 realistic valid programs in the five languages must give zero findings; the Python, Java, C and C++ ones are also checked by the real compilers (CPython, javac, gcc, g++).
- **Catch rate:** `tools/measure_accuracy.py` plants 1,601 known mistakes, one at a time, into those programs (a missing `)` or `;`, an unclosed string, `+ /`, a name starting with a digit, ...). **98.3%** are reported on the right line. A test keeps the rate above 97%.
- Every error type has a test in every language it applies to.
- Thousands of random inputs never crash the analyzer; 10,000 lines take under half a second.

## 8. Error Code Reference

| Codes | Check | Meaning |
|---|---|---|
| E101–E103 | 1 Delimiters | unclosed, unexpected or mismatched bracket |

| | | |
|---|---|---|
| E201–E204 | 2 Literals | unclosed string or char literal; empty or multi-character char literal |
| E301–E302 | 3 Terminators | missing semicolon `;`, or missing colon `:` after a Python header |
| E401–E408, W406 | 4 Operators | operators in a row, missing operand or operator, invalid assignment target, value computed but not used, empty argument |
| E501–E510, W505, W509 | 5 Headers | missing condition or parentheses, bad for/foreach header, else with a condition, bad function header; `=` in a condition, empty statement after a header |
| E601–E604 | 6 Identifiers | name starts with a digit, contains `-` or a space, is a reserved word, or is missing |
| E701–E708 | 7 Blocks | unexpected or missing indentation, mixed tabs and spaces, `else` without `if`, `break` outside a loop, `case` outside `switch` |
| E801–E805 | 8 Lexical | illegal character, malformed number, unclosed comment, curly quotes, invisible look-alike space |

Codes starting with **W** are warnings: valid code that is probably a mistake. They do not make the analysis fail.

## 9. Troubleshooting

| Problem | What to do |
|---|---|
| py is not recognized | Install Python 3.10 or newer from python.org and tick "Add python.exe to PATH". `run.bat` also tries `python` if `py` is missing. |
| The browser does not open | Look at the console window: it prints the address, e.g. `http://127.0.0.1:8000`. Open it by hand. Keep the window open while you use the app; Ctrl+C stops it. |
| Port 8000 is already in use | Nothing to do: the next free port (8001, 8002, ...) is used and printed. Or choose one: `py -m syntaxlens web --port 9000`. |
| The wrong language is detected | Very short code can look like several languages. Choose the language in the menu, or add `--lang java` on the command line. |
| "... is empty" or "looks like a binary file" | Upload the source file itself (.py, .java, .c, .cpp, .cs or .txt), not a Word document, PDF or image. |
| The online app is slow the first time | The server sleeps when nobody uses it; the first analysis wakes it up in a few seconds. The offline app has no delay. |